

Rapport de Projet — **MEDISCAN AI**

Plateforme de recherche multimodale d'images médicales par le contenu (**CBIR**)

L3X1 — Université Paris Cité



*Le présent document est le **rapport de projet** de **MEDISCAN AI**, prototype académique non clinique de recherche d'images médicales par le contenu. Il couvre l'ensemble des travaux de l'équipe L3X1 : besoins, conception, implémentation, tests et déploiement.*

Informations document

Version : v1.0
Référence : Rapport de Projet MEDISCAN AI v1.0
Confidentialité : Interne projet (usage universitaire)
Encadrant : Camille Kurtz
Équipe : L3X1 (Taskin Ozan, Rayane Taouache, Ales Ferhani, Maxime Huang)

Équipe projet

L'équipe **L3X1** est composée de quatre étudiants de l'Université Paris Cité. Chaque membre a contribué à plusieurs pôles en fonction de ses compétences.



Taskin Ozan

AI / Frontend



Ferhani Ales

AI / Test



Taouache Rayane

Backend / Test



Huang Maxime

Frontend / Backend

IA / Frontend

Taskin Ozan

Architecture du système, pipeline CBIR, fine-tuning BioMedCLIP, interface React.

IA / Test

Ferhani Ales

Scripts d'évaluation, stratégie de tests, traitement embeddings, gouvernance des données.

Backend / Test

Taouache Rayane

API FastAPI, services backend, déploiement VPS, tests unitaires.

Frontend / Backend

Huang Maxime

Interface React, pipeline et prétraitement des données, services backend, tests frontend.

Le projet a été développé **collectivement** : chaque membre a contribué à l'ensemble des composants du système. Naturellement, au fil du projet, des expertises se sont constituées au sein de l'équipe, reflétées dans la répartition des rôles ci-dessus.

Table des matières

Table des figures

Liste des tableaux

Guide de lecture

Ce rapport documente le cycle complet de réalisation de **MEDISCAN AI**, de la définition de la problématique jusqu'au déploiement en production. Plusieurs parcours de lecture sont proposés ci-dessous selon le profil du lecteur.

Le **Résumé exécutif** constitue le **point d'entrée commun à tous les lecteurs** et donne une vision globale du projet en une page.

Manager ou décideur : vision stratégique

Pour saisir la valeur du projet sans entrer dans les détails techniques.

- **Section 1** : contexte, enjeux et approche
- **Section 4** : produit déployé et cas d'usage
- **Sections 9 et 11** : bilan et perspectives

Encadrant technique : architecture et réalisation

Pour évaluer les choix techniques et la qualité de l'implémentation.

- **Section 2** : benchmark et positionnement
- **Section 5** : technologies clés et architecture
- **Section 6** : déploiement et infrastructure
- **Section 10** : impact des IA agentiques
- **Annexe A** : documentation API REST

Encadrant pédagogique : rigueur et méthode

Pour apprécier la démarche projet et la méthodologie d'évaluation.

- **Section 3** : gestion de projet et planning
- **Section 7** : évaluation et résultats
- **Section 8** : difficultés rencontrées

Lecteur externe : fonctionnalités et positionnement

Pour comprendre ce que fait le système et comment il se distingue.

- **Sections 1 et 2** : contexte et paysage concurrentiel
- **Section 4** : produit, ergonomie et cas d'usage

Le **glossaire** (Annexe D) définit les termes spécialisés du rapport.

Résumé exécutif

MEDISCAN AI est une plateforme de recherche d'images médicales par le contenu, développée par l'équipe L3X1 à l'Université Paris Cité. Elle est **aujourd'hui en production** sur <https://mediscan-ai.site>.

Contexte

Les outils actuels, notamment les systèmes **PACS**, reposent sur des mots-clés et ne permettent pas de retrouver un cas similaire à partir d'une image de référence. Les Vision Transformers récents ouvrent la voie à une recherche par similarité visuelle et sémantique.

Solution

- Moteur de recherche combinant deux approches complémentaires : similarité visuelle et analyse du contenu médical.
- Trois modes d'usage : **par image** (analyse visuelle), **par image** (sens clinique) et **par description textuelle**.
- Une synthèse automatique résume les résultats et met en évidence les tendances communes.

Architecture

- **Confidentialité** : aucune donnée ne quitte l'infrastructure de l'équipe.
- **Accessibilité** : interface web sans installation, disponible 24h/24.
- **Couverture** : index de **60 000 images médicales** référencées.

Résultats

Le système retrouve des cas pertinents et exploitables dans plus de **88 %** des situations, validé sur un jeu de **9 000 requêtes** couvrant l'ensemble des modes de recherche.

Cadre d'usage

MEDISCAN AI est un prototype académique non clinique : il ne pose aucun diagnostic et ne remplace pas un professionnel de santé.

Perspectives

- Intégration native du format **DICOM** et fine-tuning sur données internes.
- Mise en place d'un pipeline **CI/CD** pour automatiser les déploiements.

1 Contexte et enjeux

Cette section présente le cadre dans lequel s'inscrit **MEDISCAN AI**, la problématique qu'il adresse et l'approche retenue par l'équipe.

1.1 Contexte métier et motivation

La recherche d'images médicales par le contenu visuel est l'un des apports concrets des nouvelles technologies appliquées à la santé. Elle ouvre des perspectives utiles à différents profils d'utilisateurs.

- Un médecin peut retrouver rapidement des cas antérieurs similaires pour appuyer son analyse.
- Un étudiant peut explorer un corpus d'images sous différents angles pour enrichir sa formation.
- Un chercheur peut naviguer efficacement dans de grands ensembles d'images médicales.

1.2 Problématique

Comment rendre praticable la recherche d'images médicales pertinentes dans de larges corpus, sans dépendre exclusivement d'annotations manuelles et dans le respect de la confidentialité des données ?

1.3 Notre approche

Plutôt que de s'appuyer uniquement sur des mots-clés, nous avons fait le choix d'une recherche multimodale : l'utilisateur peut interroger le système par l'image ou par une description textuelle, selon son besoin.

- Deux moteurs complémentaires couvrent la similarité visuelle et le sens clinique.
- L'interface permet de comparer, filtrer et affiner les résultats sans relancer de recherche.
- Une synthèse automatique aide à interpréter les résultats retournés.

2 Benchmark et positionnement

Cette section situe **MEDISCAN AI** face aux solutions existantes en imagerie médicale.

2.1 Recherche d'images par le contenu (CBIR)

Le **CBIR** (*Content-Based Image Retrieval*) désigne la recherche d'images par leur contenu plutôt que par mots-clés. Les approches modernes fondées sur l'apprentissage profond permettent de capturer à la fois la similarité visuelle et le sens clinique d'une image.

2.2 Solutions existantes et analyse comparative

Comparaison de **MEDISCAN AI** avec les systèmes de **CBIR médical** et les principales plateformes d'imagerie médicale connues.

Système	Recherche visuelle	Recherche sémantique	Recherche textuelle	Comparaison	Synthèse IA	Interface produit	Confidentialité
MEDISCAN AI	✓	✓	✓	✓	✓	✓	✓
Yottixel	✓	✗	✗	✗	✗	✓	✓
OpenI (NLM)	✗	✗	✓	✗	✗	✓	✗
MedGemma	✗	✗	✓	✗	✓	✓	✗
MONAI (NVIDIA)	✗	✗	✗	✗	✗	✓	✓
Aidoc	✗	✗	✗	✗	✓	✓	✗

TABLE 1 – Comparaison de MEDISCAN AI avec les principales plateformes d'imagerie médicale

2.3 Un terrain d'opportunité pour la recherche

Les systèmes **PACS** déployés dans les hôpitaux ne disposent pas de recherche par le contenu visuel : les requêtes restent limitées aux métadonnées textuelles. L'intégration d'un moteur multimodal comme **MEDISCAN AI** constitue ainsi un terrain d'expérimentation prometteur pour la recherche en imagerie médicale.

3 Gestion de projet

Cette section présente la méthodologie de travail adoptée par l'équipe L3X1 et le planning du projet.

3.1 Méthodologie

L'équipe a adopté un cycle **incrémental** inspiré des méthodes agiles.

- **Incréments** : quatre livrables successifs : cœur CBIR, API backend, interface frontend, fine-tuning et évaluation.
- **Versionnement** : branches de fonctionnalité sur **SVN**, vérification avant chaque fusion sur la branche principale.
- **Coordination** : points de synchronisation bi-quotidiens sur **Discord**.

3.2 Planning et jalons

Le projet s'est déroulé sur un semestre, structuré en quatre phases successives.

- **Phase 1** : analyse des besoins, choix techniques et mise en place de l'environnement de développement.
- **Phase 2** : implémentation du cœur CBIR : embedders, index FAISS, pipeline de recherche et API backend.
- **Phase 3** : développement de l'interface React, intégration frontend-backend et fine-tuning de BioMedCLIP sur ROCov2.
- **Phase 4** : évaluation, tests, déploiement VPS et rédaction du rapport.

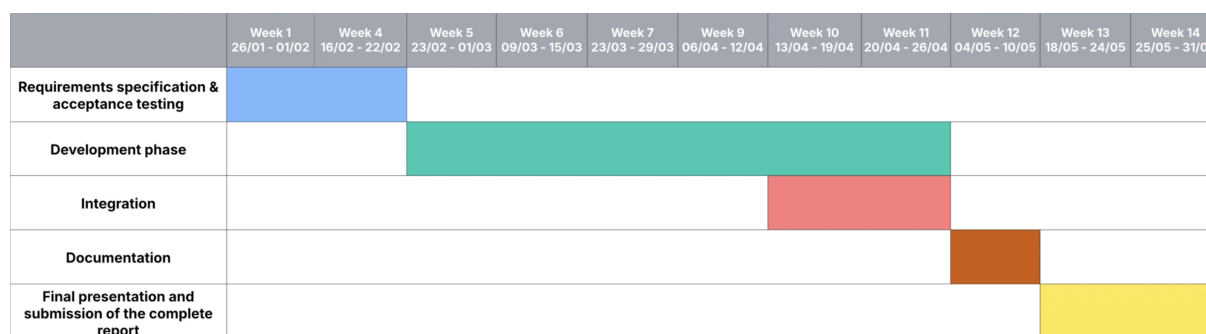


FIGURE 1 – Planning du projet MEDISCAN AI

4 Produit

Cette section présente **MEDISCAN AI** en tant que produit déployé : sa réponse concrète à la problématique posée et les cas d'usage rendus possibles par la version aujourd'hui en ligne.

4.1 Réponse à la problématique

Comment rendre praticable la recherche d'images médicales pertinentes dans de larges corpus, sans dépendre exclusivement d'annotations manuelles et dans le respect de la confidentialité des données ?

MEDISCAN AI répond à cette problématique sur trois axes complémentaires.

- **Rapidité** : optimisations sur l'index vectoriel et le traitement des données pour des réponses en quelques centaines de millisecondes.
- **Pertinence** : un corpus de référence de **60 000 images** issues du dataset public **ROCOv2**, couvrant un large spectre de modalités et d'organes.
- **Interopérabilité / Confidentialité** : architecture pluggable, capable d'indexer n'importe quel corpus interne, déployable localement ou exposée par API, sans transfert vers un service externe.

4.2 Cas d'utilisation

La plateforme est aujourd'hui en ligne sur <https://mediscan-ai.site>. Elle permet, depuis n'importe quel navigateur et sans installation, plusieurs usages concrets autour de la recherche d'images médicales.

- **Recherche par image** : retrouver les cas visuellement ou cliniquement proches d'une image requête.
- **Recherche textuelle** : interroger le corpus à partir d'une description clinique.
- **Comparaison d'images** : confronter côte à côte l'image requête et les résultats retournés.
- **Synthèse exploratoire** : obtenir une interprétation automatique des résultats à des fins d'analyse.
- **Export des résultats** : récupérer les résultats au format JSON, CSV ou PDF.

4.3 Interface et ergonomie

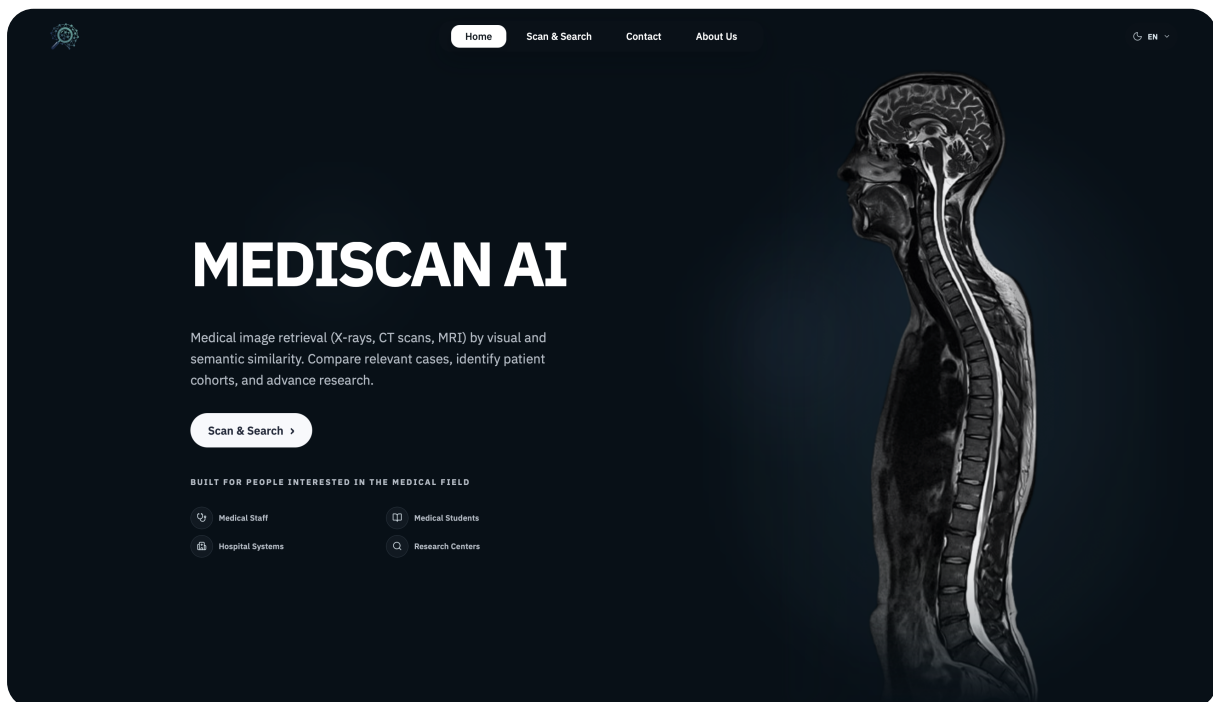


FIGURE 2 – Page d'accueil de MEDISCAN AI

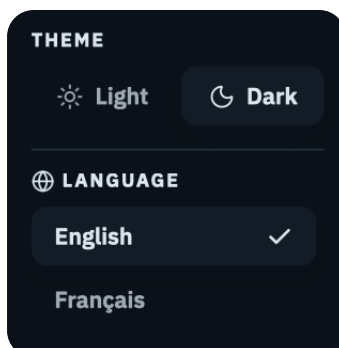


FIGURE 3 – Sélecteur de thème et de langue

L'interface propose un **thème clair et sombre** adaptable à l'environnement de travail, ainsi qu'une bascule de langue **FR / EN** accessible depuis la barre de navigation.

Le **hub central** est le point d'entrée vers les deux pages dédiées aux modes de recherche : Visual Search pour la recherche par image, et Text Guided Search pour la recherche textuelle.

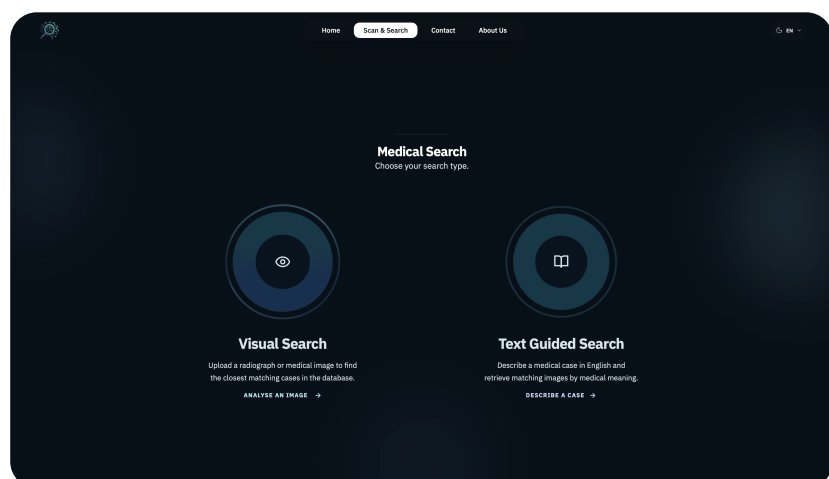
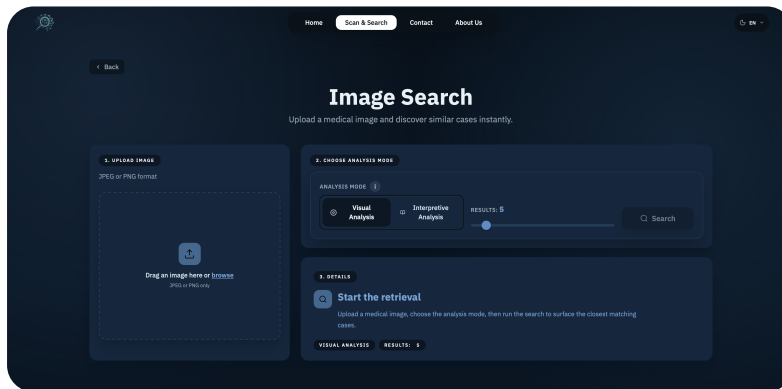


FIGURE 4 – Hub de sélection du mode de recherche

4.4 Modes de recherche

L'application propose trois modes complémentaires accessibles depuis le hub central.



L'utilisateur dépose une image médicale. Le système retourne les images visuellement proches via **DINOv2** (768D). Adapté à la recherche par morphologie et structure anatomique.

FIGURE 5 – Mode Visual Analysis

Recherche par proximité médicale via **BioMedCLIP** fine-tuné (512D). Privilégie le sens clinique sur la ressemblance visuelle.

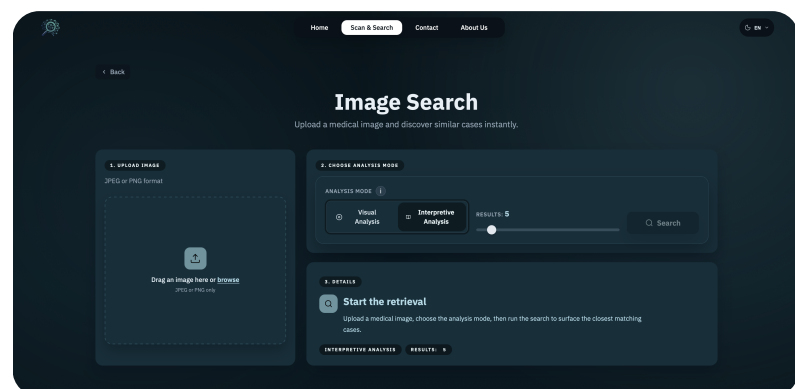
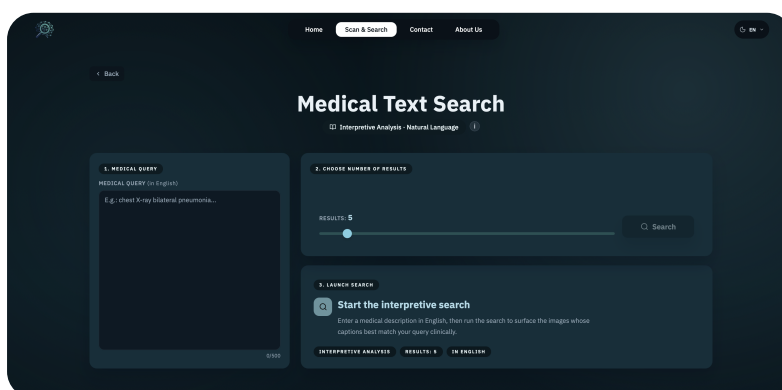


FIGURE 6 – Mode Interpretive Analysis



L'utilisateur saisit une description clinique. Le système retourne les images alignées via **BioMedCLIP** (500 caractères max).

FIGURE 7 – Mode Text Query

4.5 Résultats et exploration

Une fois la recherche lancée, l'interface propose des outils d'exploration et de relance pour affiner les résultats.

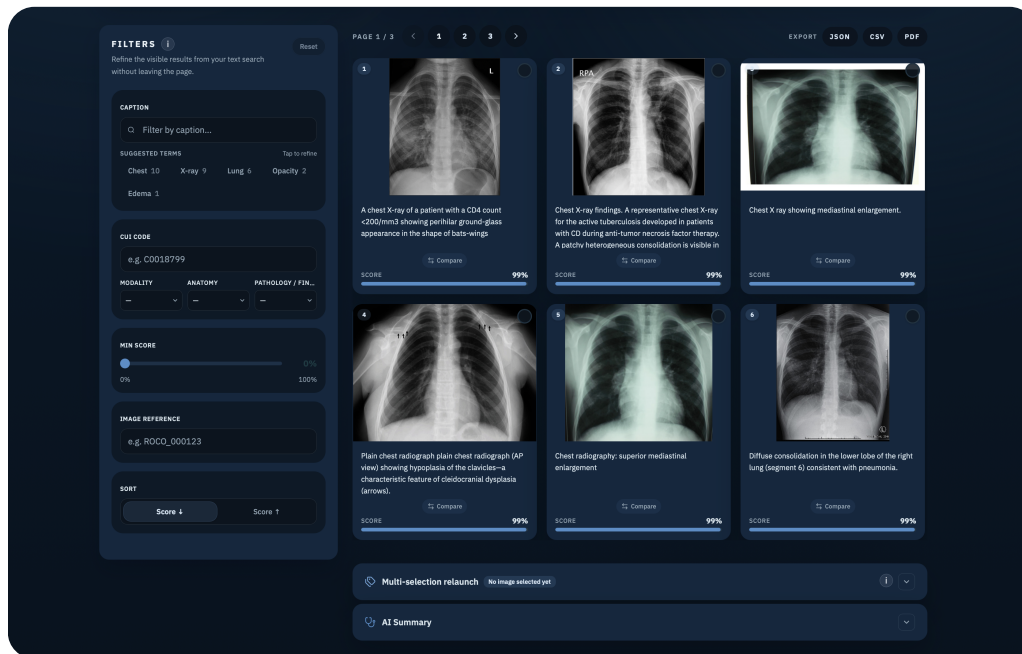


FIGURE 8 – Grille de résultats avec filtres et scores de similarité

Les filtres affinent les résultats sans relancer la recherche : **score**, **caption**, **CUI** et **tri**.

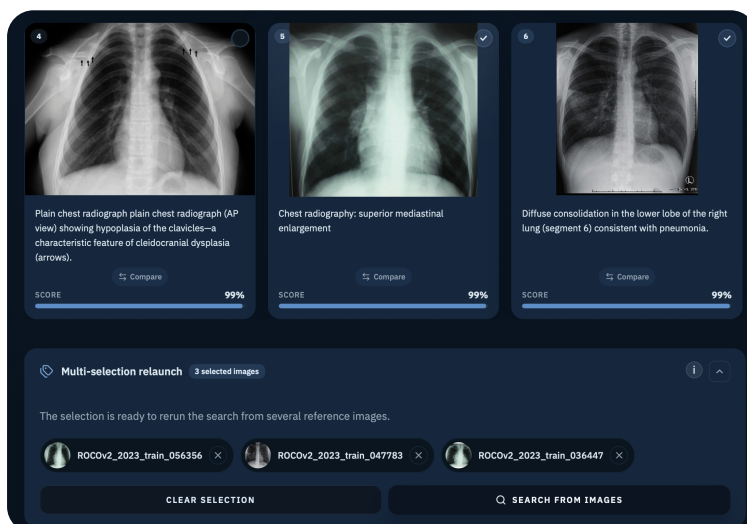


FIGURE 9 – Relance de recherche depuis une sélection d'images

Une **sélection d'images** devient une **nouvelle requête** via **centroïde d'embeddings**.

[1] Sélectionner une ou plusieurs images.

[2] Relancer depuis la sélection.

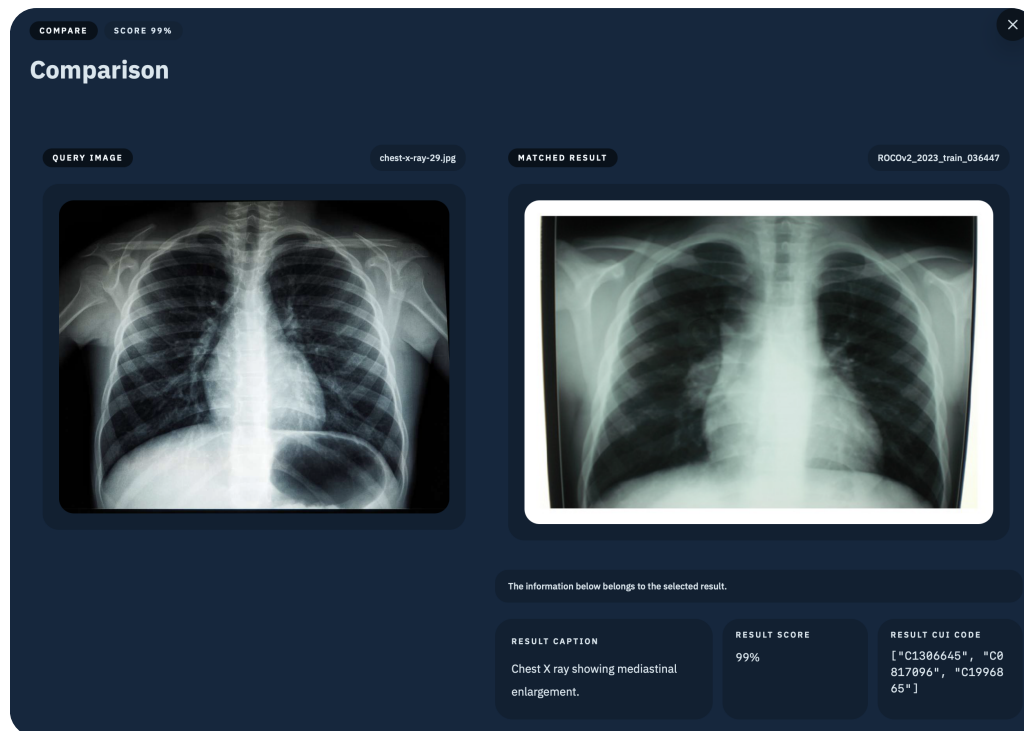


FIGURE 10 – Comparaison entre l'image requête et les résultats retournés

L'interface affiche **côte à côte** l'**image requête** et les **résultats retournés**, permettant une **comparaison visuelle directe**.

Le **module de synthèse** analyse les résultats et génère une **interprétation exploratoire** assistée par IA.

- [1] Synthèse générée via **Groq** à partir des captions des résultats.
- [2] Identifie les **patterns récurrents** sans poser de diagnostic.

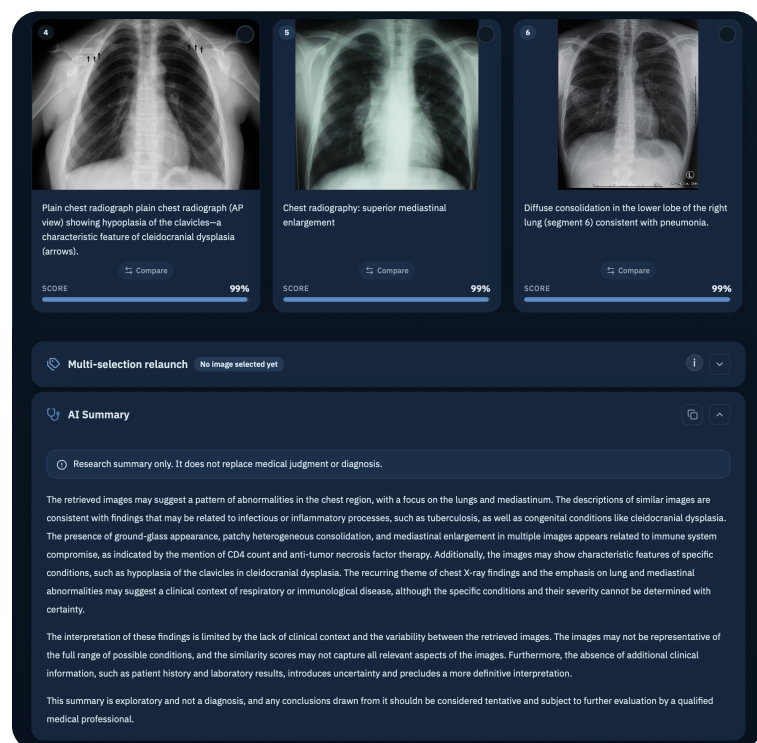


FIGURE 11 – Synthèse exploratoire générée par IA

5 Technologies clés

MEDISCAN AI s'appuie sur deux modèles d'IA aux architectures complémentaires, combinés à un moteur de recherche vectorielle haute performance.

5.1 Architecture des pipelines d'embedding

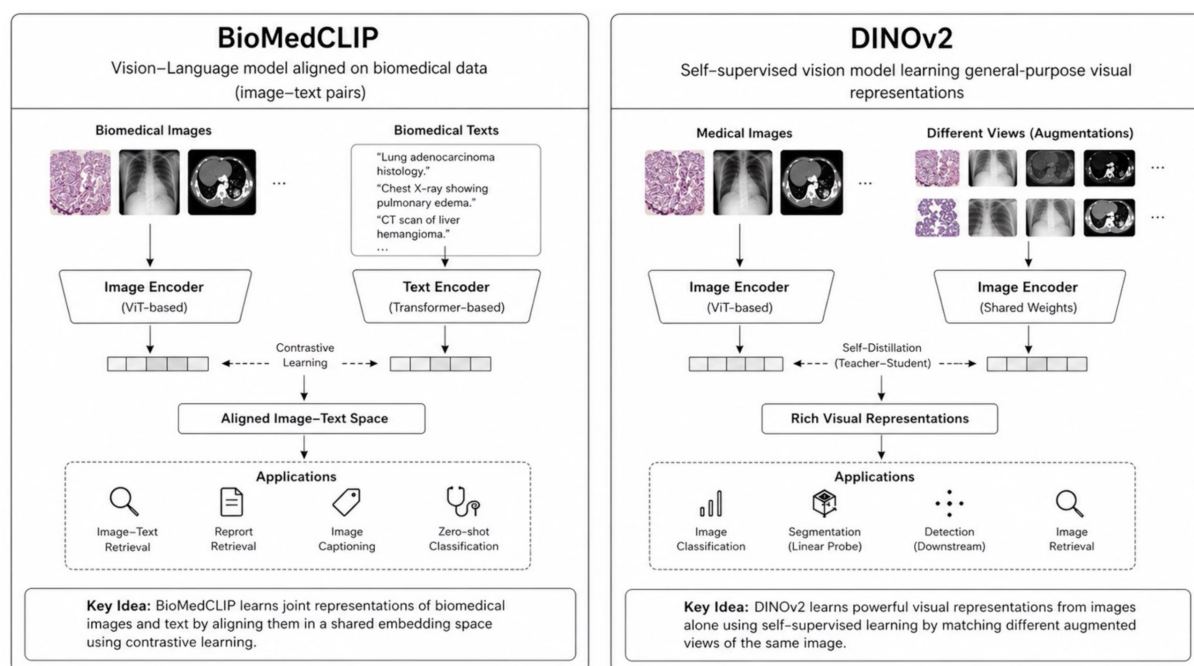


FIGURE 12 – Architecture des deux pipelines d'embedding — BioMedCLIP (sémantique) et DINOv2 (visuel)

BioMedCLIP (512D)

Architecture duale : un encodeur image (ViT) et un encodeur texte (Transformer) entraînés conjointement sur 15 millions de paires image-texte biomédicales.

- Aligne images et textes dans un espace vectoriel commun.
- Utilisé pour les modes **Interpretive Analysis** et **Text Guided Search**.

DINOv2 (768D)

Vision Transformer (ViT-L/14) pré-entraîné de manière auto-supervisée sur 142 millions d'images sans annotation.

- Capture structures visuelles, textures et morphologie.
- Utilisé pour le mode **Visual Search**.

Fine-tuning de BioMedCLIP. Le modèle a été ré-entraîné sur le dataset **ROCOv2** pendant plusieurs epochs afin d'y adapter ses représentations.

5.2 Technologies de développement

Stack moderne organisée en trois blocs : frontend, backend et couche IA.



5.3 FAISS et optimisation de la recherche

La recherche vectorielle repose sur **FAISS** (*Facebook AI Similarity Search*).

- Index **FlatIP** sur **60 000 vecteurs** : recherche exacte en quelques ms.
- Chargement *lazy* des modèles et de l'index pour limiter la mémoire.
- Architecture *stateless* : passage à l'échelle vers plusieurs millions d'images.

5.4 Stockage cloud et infrastructure d'entraînement

Deux ressources externes complémentaires : **Hugging Face** pour les données, **Kaggle** pour le calcul.



Hugging Face : Stockage cloud

Plateforme essentielle qui héberge l'ensemble des **60 000 images** du dataset **ROCOv2**. Elle permet un accès à la demande, sans dupliquer le corpus en local, tout en garantissant l'unicité de la version utilisée par l'équipe et un déploiement allégé.



Kaggle : GPU d'entraînement

Ressource clé pour le **fine-tuning de BioMedCLIP**. Les GPU mis à disposition gratuitement par Kaggle ont permis d'entraîner les modèles sans investir dans une infrastructure matérielle dédiée, ouvrant la voie à des itérations rapides.

6 Déploiement et exploitation

Cette section présente l'infrastructure mise en place pour rendre **MEDISCAN AI** accessible en production, ainsi que les choix garantissant sa stabilité et son évolutivité.

6.1 Architecture et pipeline de déploiement

L'application est déployée sur un **VPS Hostinger**, accessible via `mediscan-ai.site`.

- **Nginx** - reverse proxy, routage des requêtes frontend et backend.
- **Gunicorn + Uvicorn** - serveur ASGI pour l'API FastAPI.
- **systemd** - gestion des services et redémarrage automatique.

Le déploiement suit un pipeline en quatre étapes.

- [1] Configuration du serveur Linux : mises à jour, firewall, ports 80/443.
- [2] Installation des dépendances backend et build du frontend (Vite).
- [3] Configuration Nginx avec virtual host et certificat SSL.
- [4] Activation des services systemd pour le backend et le frontend.

6.2 Production, accessibilité et évolutivité

MEDISCAN AI est déployé en production et utilisable depuis n'importe quel navigateur, sans installation côté utilisateur.

- **Accessibilité** - le service est disponible 24h/24 via `mediscan-ai.site`, avec connexion chiffrée HTTPS.
- **Confidentialité** - aucune image n'est transmise à un service externe ; tout le traitement s'effectue sur le serveur de l'équipe.
- **Stabilité** - le redémarrage automatique (systemd) garantit la continuité du service en cas d'arrêt inattendu.

L'architecture choisie permet une montée en charge progressive.

- L'index FAISS est conçu pour passer de 60 000 à plusieurs millions d'images sans modifier l'interface ni l'API.
- Le backend FastAPI est sans état (*stateless*) : plusieurs instances peuvent être déployées en parallèle derrière Nginx pour absorber la charge.
- L'ajout d'un nouveau mode de recherche ou d'un nouveau modèle d'embedding ne nécessite pas de refonte de l'architecture.

7 Évaluation et résultats

L'équipe a pu s'appuyer sur des **annotations manuelles** produites par des **doctorants en imagerie médicale**, fournissant une vérité terrain experte (**modalité, organe, région anatomique**). Plus de **9 000 requêtes** ont ainsi été évaluées sur les deux moteurs.

7.1 Résultats du moteur visuel (DINOv2)

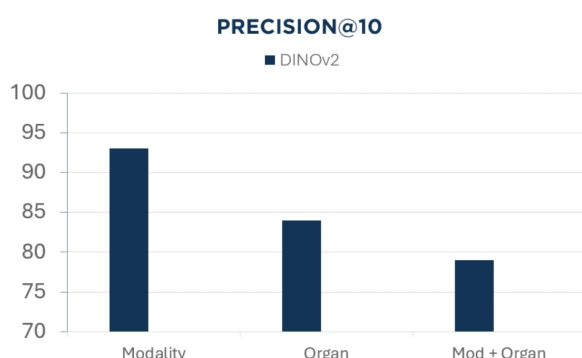


FIGURE 13 – DINOv2 : Precision@k

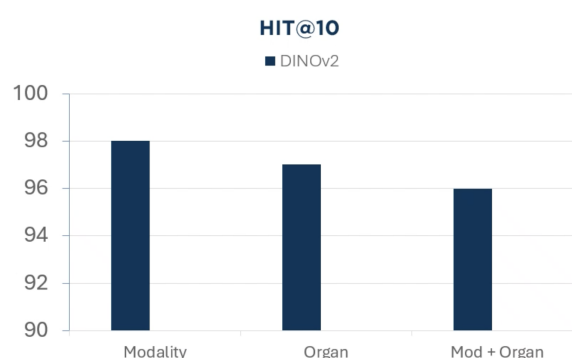


FIGURE 14 – DINOv2 : Hit@k

Interprétation. DINOv2 obtient une **Precision** élevée sur la modalité et l'anatomie, confirmant qu'il capture efficacement les structures visuelles. Le **Hit@k** valide son usage pour la recherche par similarité visuelle.

7.2 Résultats du moteur sémantique (BioMedCLIP)

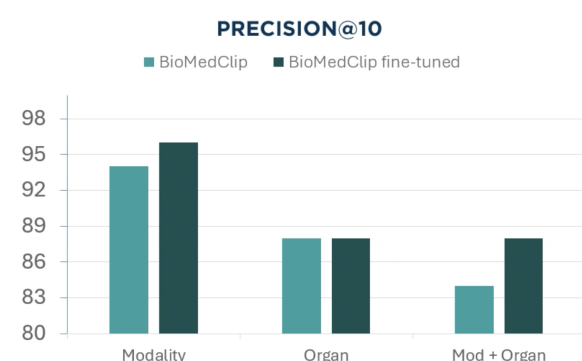


FIGURE 15 – BioMedCLIP : Precision@k

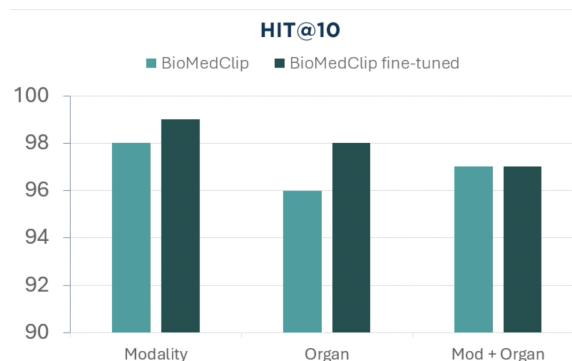


FIGURE 16 – BioMedCLIP : Hit@k

Interprétation. BioMedCLIP capte mieux le **sens clinique** grâce à son alignement image-texte fine-tuné sur ROCov2. Ses scores combinant modalité et organe sont supérieurs à DINOv2, validant son usage pour la recherche sémantique.

7.3 Tests unitaires

Une stratégie de tests **unitaires** et **d'intégration** a été mise en place pour garantir la fiabilité du backend et la stabilité du pipeline de recherche.

- **Backend** : tests `pytest` couvrant les neuf endpoints REST, validation des entrées, formats de réponse, gestion des erreurs.
- **Pipeline CBIR** : tests sur les embedders, le chargement des index **FAISS** et la cohérence des résultats de recherche.
- **Intégration** : scénarios bout-en-bout simulant une requête utilisateur depuis l'upload d'image jusqu'au retour des résultats.

7.4 Performance technique

Au-delà de la pertinence des résultats, le système a été évalué sur ses **performances en production**.

- **Temps de réponse** : recherche vectorielle exécutée en **quelques centaines de millisecondes** sur l'index de 60 000 vecteurs.
- **Empreinte mémoire** : chargement *lazy* des modèles et de l'index pour limiter la consommation au démarrage.
- **Scalabilité** : architecture *stateless* permettant de déployer plusieurs instances backend derrière Nginx pour absorber la charge.

7.5 Limites observées et synthèse

- **Modalités rares** : les modalités sous-représentées dans ROCov2 obtiennent des scores plus faibles que les modalités majoritaires.
- **Captions ambiguës** : la recherche textuelle est sensible à la qualité et à la précision de la description fournie.
- **Dépendance au dataset** : les performances reflètent le périmètre de ROCov2 et ne préjugent pas du comportement sur d'autres corpus médicaux.

Synthèse. Malgré ces limites, les résultats valident la pertinence de **MEDISCAN AI** : les deux moteurs sont **complémentaires** (DINOv2 pour la proximité visuelle, BioMedCLIP pour le sens clinique), les tests garantissent la fiabilité du système et les performances permettent un usage fluide en production.

8 Difficultés rencontrées

Cette section revient sur les principales difficultés rencontrées au cours du projet.

8.1 Fine-tuning et choix des modèles

Plusieurs candidats existaient pour la partie sémantique (BioMedCLIP, PubMedCLIP, MedCLIP).

- **Choix du modèle** : sélection de **BioMedCLIP** pour son alignement image-texte biomédical.
- **Fine-tuning** : adaptation à **ROCOv2** avec plusieurs itérations d'hyperparamètres.
- **Ressources GPU limitées** : entraînement sur les GPU **Kaggle**, avec des quotas contraints.

8.2 Manipulation d'un dataset volumineux

Les **60 000 images** de ROCov2 ont soulevé plusieurs défis de volumétrie.

- **Stockage et bande passante** : dépendance à **Hugging Face**, téléchargements non négligeables.
- **Génération des embeddings** : opération longue et coûteuse en mémoire sur les deux modèles.
- **Manque de ressources cloud** : pas de cluster GPU dédié, contraintes mémoire sur le VPS.

8.3 Absence de benchmark de référence en CBIR médical

Aucun **benchmark standardisé** n'existe pour évaluer un système CBIR médical multimodal.

- **Méthodologie ad hoc** : démarche d'évaluation construite à partir des métadonnées.
- **Annotations expertes** : mobilisation de **doctorants en imagerie médicale** pour la vérité terrain.
- **Comparabilité** : difficulté à confronter les résultats à d'autres systèmes.

9 Bilan et retour d'expérience

Cette section dresse un bilan du projet, à la fois sur les acquis techniques et sur l'expérience humaine vécue par l'équipe au cours du semestre.

9.1 Réussites techniques

Le projet a abouti à une plateforme fonctionnelle, complète et déployée en production, accessible publiquement à l'adresse `mediscan-ai.site`.

- **Multimodalité** : intégration réussie de la recherche visuelle, sémantique et textuelle dans une seule interface cohérente.
- **Mise en production** : déploiement complet sur VPS avec **Nginx**, **SSL** et services `systemd` pour garantir la stabilité du service.
- **Fine-tuning** : adaptation effective de **BioMedCLIP** sur ROCov2, avec des résultats validés par les doctorants en imagerie médicale.

9.2 Apports méthodologiques

Le projet a permis de structurer une démarche complète d'ingénierie logicielle appliquée à un cas concret d'intelligence artificielle.

- **Cycle incrémental** : découpage en quatre livrables successifs, du cœur CBIR au déploiement final.
- **Évaluation rigoureuse** : construction d'une méthodologie d'évaluation appuyée sur des annotations expertes et plus de 9 000 requêtes.
- **Versionnement collaboratif** : usage quotidien de **Git** avec branches de fonctionnalités, revues croisées et historique propre.

9.3 Apports humains et travail d'équipe

Au-delà des aspects techniques, le projet a constitué une véritable expérience de collaboration et d'apprentissage collectif.

- **Spécialisation progressive** : chaque membre a développé une expertise reconnue (IA, frontend, backend, tests).
- **Coordination** : points de synchronisation réguliers sur **Discord** pour partager les blocages et avancées.
- **Polyvalence** : contribution croisée de chaque membre à plusieurs composants du système, renforçant la cohésion d'équipe.

10 Impact agentic AI

Cette section revient sur l'usage des IA agenticues dans la réalisation du projet et sur leur apport à la productivité de l'équipe.

10.1 État actuel des IA agenticues

Les modèles d'IA dits **agenticues** ont connu une évolution rapide en 2024-2025, passant de simples assistants conversationnels à de véritables collaborateurs capables d'agir sur du code, des fichiers et des outils.

- **Claude (Anthropic)** et **Codex (OpenAI)** dominent les usages développeurs avec leurs capacités d'écriture, de relecture et de refactorisation de code.
- Les agents savent désormais **exécuter des commandes**, lire des logs, modifier plusieurs fichiers et raisonner sur un projet entier.

10.2 Utilisation dans le projet

L'équipe a intégré ces outils à différentes étapes du développement, de la conception à la rédaction du rapport, à travers des prompts ciblés.

- **Génération de code** : « *Écris un endpoint FastAPI qui retourne les k plus proches voisins via FAISS à partir d'une image uploadée.* »
- **Debug et refactorisation** : « *Pourquoi ce test pytest échoue ? Propose un correctif.* »
- **Rédaction et structuration** : « *Rédige la section Évaluation en t'appuyant sur les graphiques DINOv2 et BioMedCLIP.* »

10.3 Apports et limites

L'usage des IA agenticues a transformé la manière de travailler, sans pour autant remplacer le jugement humain.

- **Avantages** : accélération du développement, exploration rapide de pistes techniques, qualité de code améliorée et gain de temps sur les tâches répétitives.
- **Limites** : nécessité de relire systématiquement les sorties, risques d'hallucinations sur des contextes pointus, dépendance à un service externe.
- **Posture adoptée** : l'IA reste un assistant ; chaque proposition est validée, testée et adaptée par l'équipe avant intégration.

11 Perspectives

Plusieurs pistes d'évolution se dessinent pour prolonger et enrichir **MEDISCAN AI**.

11.1 Évolutions fonctionnelles

La plateforme peut s'enrichir de nouvelles capacités côté utilisateur.

- **Support du format DICOM** : ingestion native des images médicales au format standard hospitalier.
- **Recherche par filtres avancés** : combinaison de critères cliniques (modalité, organe, pathologie).
- **Export enrichi** : génération de rapports PDF combinant images, scores et synthèse IA.

11.2 Évolutions techniques

L'infrastructure et les modèles peuvent être renforcés pour passer à l'échelle.

- **Pipeline CI/CD** : automatisation des tests, du build et du déploiement à chaque commit.
- **Index FAISS optimisé** : passage d'un index **FlatIP** à un index **IVF** ou **HNSW** pour des volumes supérieurs.
- **Fine-tuning continu** : ré-entraînement régulier sur de nouvelles données pour améliorer la pertinence.

11.3 Vers la recherche et l'industrie

MEDISCAN AI ouvre des perspectives concrètes d'usage au-delà du cadre académique.

- **Intégration aux systèmes PACS** : brancher le moteur multimodal sur les bases hospitalières existantes.
- **Partenariats cliniques** : collaboration avec des établissements pour valider l'usage en conditions réelles.
- **Publications et benchmarks** : contribuer à combler le manque de standards d'évaluation en CBIR médical.

Références bibliographiques

- [1] M. Caron *et al.*, “Emerging Properties in Self-Supervised Vision Transformers,” *ICCV*, 2021.
- [2] S. Zhang *et al.*, “BioMedCLIP : a multimodal biomedical foundation model,” *arXiv :2303.00915*, 2023.
- [3] J. Johnson, M. Douze, H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, 2019.
- [4] FastAPI Documentation. <https://fastapi.tiangolo.com>
- [5] React Documentation. <https://react.dev>

A Documentation API REST

Les neuf endpoints de l'API sont exposés sous la base `/api/v1`.

Méthode	Endpoint	Description
POST	<code>/search/visual</code>	Recherche visuelle par image (DINOv2).
POST	<code>/search/semantic</code>	Recherche sémantique par image (BioMedCLIP).
POST	<code>/search/text</code>	Recherche par description textuelle (BioMedCLIP).
POST	<code>/search/multisel</code>	Relance depuis une sélection d'images (centroïde).
GET	<code>/synthesis</code>	Génère la synthèse IA pour un ensemble de résultats.
GET	<code>/export/json</code>	Export des résultats au format JSON.
GET	<code>/export/csv</code>	Export des résultats au format CSV.
GET	<code>/export/pdf</code>	Export des résultats au format PDF.
GET	<code>/health</code>	Vérification de l'état du service.

TABLE 2 – Endpoints REST exposés par l'API MEDISCAN AI

B Extraits de code et utilisation de l'IA

Cette annexe présente les extraits de code les plus significatifs du projet MEDISCAN AI, illustrant les choix d'implémentation des composants centraux du système : les encodeurs d'images, le moteur de recherche FAISS, et l'utilisation de l'intelligence artificielle au cours du développement.

B.1 Encodeur BioMedCLIP

L'encodeur BioMedCLIP est le composant central du mode Analyse Interprétative. Il encode les images et les requêtes textuelles dans un espace vectoriel commun de dimension 512, permettant la recherche sémantique multimodale. Le modèle utilisé est une version fine-tunée sur ROCov2, hébergée sur HuggingFace sous l'identifiant `Ozantask/biomedclip-rocov2-finetuned`.

Listing 1 – Implémentation de l'encodeur BioMedCLIP
(src/mediscan/embedders/biomedclip.py)

```

1 class BioMedCLIPEmbedder(Embedder):
2     """
3     BioMedCLIP image and text embedder for semantic medical search.
4     The image and text encoders share the same vector space, allowing
5     text prompts and medical images to query the same semantic FAISS
6     index.
7     """

```

```

7     name = "biomedclip"
8     dim = 512
9
10    def __init__(
11        self,
12        model_name: str = "hf-hub:0zantsk/biomedclip-rocov2-finetuned",
13    ) -> None:
14        configure_torch_cpu_threads()
15        self._device = torch.device("cpu")
16        self._model, _, self._preprocess = open_clip.
17            create_model_and_transforms(
18                model_name
19            )
20        self._model.to(self._device)
21        self._model.eval()
22        self._tokenizer = open_clip.get_tokenizer(model_name)
23
24    def encode_pil(self, image: PILImage.Image) -> np.ndarray:
25        """Encode a PIL image into the normalized semantic vector space.
26        """
27        rgb_image = image.convert("RGB")
28        input_tensor = self._preprocess(rgb_image).unsqueeze(0).to(self.
29            _device)
30        with torch.no_grad():
31            features = self._model.encode_image(input_tensor)
32        return normalize_embedding(features.squeeze(0).cpu().numpy(),
33            self.dim)
34
35    def encode_text(self, text: str) -> np.ndarray:
36        """Encode a text query into the same normalized space as image
37        embeddings."""
38        tokens = self._tokenizer([text]).to(self._device)
39        with torch.no_grad():
40            features = self._model.encode_text(tokens)
41        return normalize_embedding(features.squeeze(0).float().cpu().
42            numpy(), self.dim)

```

B.2 Encodeur DINOv2

L'encodeur DINOv2 est le composant central du mode Analyse Visuelle. Il produit des vecteurs de dimension 768 capturant la structure anatomique et visuelle des images médicales, sans avoir été entraîné sur des données médicales. Le *pooler output* est utilisé en priorité ; le token CLS est utilisé comme alternative si le modèle ne l'expose pas.

Listing 2 – Implémentation de l'encodeur DINOv2 (src/mediscan/embedders/dinov2_base.py)

```

1 class DINOv2BaseEmbedder(Embedder):
2     """
3     DINOv2 base image embedder for visual similarity search.
4     Compares image content directly without text supervision at query
5     time.
6     """
7     name = "dinov2_base"
8     dim = 768
9
10    def __init__(self, model_name: str = "facebook/dinov2-base") -> None
11    :
12        configure_torch_cpu_threads()
13        self._device = torch.device("cpu")
14        self._processor = AutoImageProcessor.from_pretrained(
15            model_name, use_fast=True
16        )
17        self._model = AutoModel.from_pretrained(model_name)
18        self._model.to(self._device)
19        self._model.eval()
20
21    def encode_pil(self, image: PILImage.Image) -> np.ndarray:
22        """Encode a PIL image into a normalized visual embedding vector.
23        """
24
25        rgb_image = image.convert("RGB")
26        inputs = self._processor(images=rgb_image, return_tensors="pt")
27        pixel_values = inputs["pixel_values"].to(self._device)
28
29        with torch.no_grad():
30            outputs = self._model(pixel_values=pixel_values)
31
32        # Pooler output preferred; CLS token used as fallback
33        if getattr(outputs, "pooler_output", None) is not None:
34            features = outputs.pooler_output
35        else:
36            features = outputs.last_hidden_state[:, 0]
37
38        return normalize_embedding(features.squeeze(0).cpu().numpy(),
39                                self.dim)

```

B.3 Moteur de recherche FAISS

Le moteur de recherche centralise le chargement des ressources, l'encodage des requêtes et la recherche dans l'index FAISS. Trois types de requêtes sont supportés : par image uploadée, par identifiant d'image déjà indexée, et par texte médical.

Listing 3 – Recherche par image uploadée (src/mediscan/search.py)

```

1 def query(
2     *,
3     resources: SearchResources,
4     image: str | Path,
5     k: int,
6     exclude_self: bool = False,
7 ) -> list[dict[str, Any]]:
8     """Encode a query image and search the matching FAISS index."""
9     _validate_k(k)
10    query_image = resolve_path(image)
11
12    with Image.open(query_image) as pil_image:
13        query_vector = (
14            resources.embedder.encode_pil(pil_image)
15            .reshape(1, -1)
16            .astype(np.float32)
17        )
18    faiss.normalize_L2(query_vector)
19
20    search_k = compute_search_k(k, resources.index.ntotal, exclude_self=
        exclude_self)
21    scores, indices = resources.index.search(query_vector, search_k)
22
23    excluded_ids = {query_image.stem} if exclude_self else set()
24    excluded_paths = {str(query_image.resolve())} if exclude_self else
        set()
25
26    return collect_ranked_results(
27        rows=resources.rows,
28        scores=scores[0],
29        indices=indices[0],
30        k=k,
31        excluded_image_ids=excluded_ids,
32        excluded_paths=excluded_paths,
33    )

```

Listing 4 – Recherche par texte médical (src/mediscan/search.py)

```

1 def query_text(
2     *,
3     resources: SearchResources,
4     text: str,
5     k: int,
6 ) -> list[dict[str, Any]]:
7     """Encode a text prompt and search the semantic FAISS index."""
8     _validate_k(k)

```



```
9     if not hasattr(resources.embedder, "encode_text"):
10         raise ValueError(
11             "Embedder does not support encode_text(). "
12             "Use mode='semantic' (BioMedCLIP) for text queries."
13         )
14
15     query_vector = (
16         resources.embedder.encode_text(text.strip())
17         .reshape(1, -1)
18         .astype(np.float32)
19     )
20     faiss.normalize_L2(query_vector)
21
22     search_k = compute_search_k(k, resources.index.ntotal)
23     scores, indices = resources.index.search(query_vector, search_k)
24
25     return collect_ranked_results(
26         rows=resources.rows,
27         scores=scores[0],
28         indices=indices[0],
29         k=k,
30     )
```

B.4 Utilisation de l'intelligence artificielle

Au cours du développement de MEDISCAN AI, des outils d'intelligence artificielle générative ont été utilisés à plusieurs étapes du projet : génération de code, débogage, rédaction de documentation et aide à la conception. Les exemples ci-dessous illustrent quelques-uns des prompts rédigés par l'équipe.

Implémente une classe Python BioMedCLIPEmbedder qui charge le modèle hf-hub:Ozantask/biomedclip-rocov2-finetuned via open_clip, expose une méthode encode_pil pour encoder une image PIL, et une méthode encode_text pour encoder une requête textuelle. Les deux méthodes doivent retourner un vecteur numpy normalisé L2 de dimension 512.

Ce prompt a servi de base à l'implémentation de l'encodeur BioMedCLIP, ensuite adaptée et intégrée au pipeline MEDISCAN AI.

Comment implémenter une recherche de voisins les plus proches avec FAISS IndexFlatIP sur des embeddings normalisés L2 de dimension 768 ? La fonction doit accepter un vecteur requête numpy, retourner les k résultats les plus similaires avec leurs scores, et exclure l'image requête elle-même des résultats si elle est déjà indexée.

Ce prompt a guidé l'implémentation du moteur de recherche vectorielle, notamment la gestion de l'exclusion de la requête et du paramètre `search_k`.

Écris un script Python d'évaluation qui mesure la Precision@k d'un index FAISS sur trois critères : même modalité (TM), même organe (TA), et même modalité et organe (TMO), à partir de fichiers CSV de ground truth. Le script doit accepter les arguments `-mode`, `-k`, `-n-queries` et `-seed`, et afficher la progression toutes les 500 requêtes.

Ce prompt a fourni la structure du script `evaluate_strict.py`, qui a ensuite été enrichi avec la gestion du pool de candidats et l'export CSV.

Prouve mathématiquement qu'il est impossible de reconstruire une image médicale depuis son embedding DINOv2 de dimension 768. Calcule le ratio de compression pour une image de 682×748 pixels et explique pourquoi la perte d'information est irréversible.

Ce prompt a permis de formaliser l'argument de confidentialité des données présenté dans la gouvernance du système et dans la politique de confidentialité de l'application.

C Résultats de tests détaillés

Les évaluations ont été conduites sur un sous-ensemble de 200 images requêtes pour les modes visuel et sémantique, et sur 100 images pour la recherche par texte, échantillonnées aléatoirement avec la graine 42 depuis le ground truth disponible. Trois critères de pertinence ont été définis : TM (même modalité), TA (même organe), et TMO (même modalité et même organe). La métrique principale est la
Precision@10.

C.1 Mode Analyse Visuelle (DINOv2)

Sur 200 requêtes évaluées avec $k = 10$, le mode Analyse Visuelle obtient une Precision@10 de **93,25%** sur le critère TM, **37,40%** sur le critère TA, et **36,00%** sur le critère TMO. Ces résultats montrent que DINOv2 est particulièrement efficace pour retrouver des images de même modalité d'imagerie, ce qui est cohérent avec sa nature de modèle visuel généraliste capturant la structure globale des images. En revanche, la précision sur l'organe seul ou la combinaison modalité-organe est plus faible, ce qui s'explique par le fait que DINOv2 n'a jamais été entraîné sur des données médicales annotées.

Critère	Precision@10
TM — Même modalité	93,25%
TA — Même organe	37,40%
TMO — Modalité + organe	36,00%

C.2 Mode Analyse Interprétative (BioMedCLIP)

Sur les mêmes 200 requêtes avec $k = 10$, le mode Analyse Interprétative obtient une Precision@10 de **93,45%** sur TM, **46,80%** sur TA, et **44,10%** sur TMO. BioMedCLIP surpasse DINOv2 sur les critères TA et TMO, avec un gain de +9,40 points sur l'organe et +8,10 points sur la combinaison modalité-organe. Ce résultat confirme l'apport du fine-tuning sur ROCov2 et de l'alignement image-texte médical pour capturer la sémantique clinique des images au-delà de leur apparence visuelle.

Critère	Precision@10
TM — Même modalité	93,45%
TA — Même organe	46,80%
TMO — Modalité + organe	44,10%

C.3 Mode Recherche par Texte (BioMedCLIP)

L'évaluation de la recherche par texte a été conduite sur 100 images en deux modes : *caption* (légende complète utilisée comme requête) et *keyword* (mots-clés extraits de la légende).

En mode **caption**, le système atteint une Precision@10 de **98,10%**, un Top-1 hit de **17,00%** et un Recall@10 de **49,00%**. La Precision@10 très élevée indique que les résultats retournés sont quasi-systématiquement pertinents. Le Top-1 hit de 17% signifie que dans 17% des cas, l'image source elle-même est classée en première position, ce qui est remarquable compte tenu de la taille de la base.

En mode **keyword**, la Precision@10 reste solide à **94,50%**, mais le Top-1 hit tombe à **0,00%** et le Recall@10 à **7,00%**. Ce résultat est attendu : une requête réduite à quelques mots-clés est une représentation moins précise de l'image source, ce qui rend le rappel de l'image exacte beaucoup plus difficile. Néanmoins, la haute précision montre que même avec une requête partielle, le système retrouve des images cliniquement pertinentes.

Mode	Precision@10	Top-1 hit	Recall@10
Caption	98,10%	17,00%	49,00%
Keyword	94,50%	0,00%	7,00%

D Glossaire

BioMedCLIP	Modèle multimodal pré-entraîné sur des données biomédicales, alignant images et textes dans un espace vectoriel commun de dimension 512.
CBIR	<i>Content-Based Image Retrieval</i> — recherche d'images par le contenu visuel ou sémantique, sans recours à des métadonnées textuelles manuelles.
Centroïde	Vecteur moyen de plusieurs embeddings, utilisé pour la relance multi-sélection afin de représenter la tendance commune d'un groupe.
CUI	<i>Concept Unique Identifier</i> — identifiant unique d'un concept dans la nomenclature médicale UMLS.
DINOv2	Modèle de vision auto-supervisé développé par Meta AI, utilisé pour extraire des représentations visuelles de dimension 768.
Embedding	Représentation vectorielle d'une image ou d'un texte dans un espace de grande dimension, calculée par un réseau de neurones.
FAISS	<i>Facebook AI Similarity Search</i> — bibliothèque de recherche de voisins les plus proches dans des espaces vectoriels de grande dimension.
FastAPI	Framework Python asynchrone pour la création d'API REST, utilisé comme couche applicative backend du système.
Fine-tuning	Adaptation d'un modèle pré-entraîné à un domaine spécifique par un entraînement complémentaire sur des données ciblées.
Groq	Service d'inférence LLM rapide, utilisé pour la génération de la synthèse assistée.
LLM	<i>Large Language Model</i> — modèle de langage de grande taille, utilisé ici pour générer une synthèse exploratoire à partir des résultats.
MongoDB	Base de données orientée documents, utilisée pour l'enrichissement optionnel des métadonnées associées aux résultats de recherche.
React	Bibliothèque JavaScript pour la construction d'interfaces utilisateur, utilisée pour le frontend du système.
ROCOv2	<i>Radiology Objects in Context v2</i> — dataset public d'images médicales annotées avec des captions et des concepts UMLS (CUI).
Top-k	Les k résultats les plus similaires à une requête, retournés par ordre décroissant de similarité.
UMLS	<i>Unified Medical Language System</i> — système de terminologie médicale unifié, référence pour les concepts cliniques.

Vite Outil de build et de développement frontend rapide, utilisé pour compiler et servir l'application React.